DAYTONE MOOD ANALYSER

Academic Progress Presentation Guide & Development Roadmap

Executive Summary: DayTone is a production-ready, GDPR-compliant Flask web application for personal mood tracking and machine learning-based burnout prediction. Designed to protect sensitive data, it utilizes local natural language processing (NLTK VADER) and on-device ML models (Decision Tree, Random Forest) to evaluate wellness metrics (mood, sleep, stress, activity, social log) and predict risk. This guide provides a monthly and weekly blueprint for academic evaluations, detailing exactly what to implement, demonstrate, and say during reviews, along with anticipated Viva Voce questions.

Candidate Name:	S A Sanush
Academic Year:	2026 (Semester 3)
Course Program:	Master of Computer Applications (MCA)
Project Category:	Semester 3 Mini Project (Final Submission: Oct/Nov 2026)
Core Architecture:	Flask, PostgreSQL, Redis, Scikit-learn, NLTK VADER, Three.js
Deployment URL:	https://daytone-piyi.onrender.com

1. Semester Milestones & Month-by-Month Roadmap

To satisfy the academic requirements of a Semester 3 MCA project, development is broken down into structured, cumulative monthly phases. This structure demonstrates solid software engineering practices, moving from requirement engineering to database design, core backend logic, frontend dashboards, machine learning training, monitoring, and production deployment.

Month / Milestone	Core Objectives & Implementation Focus	Review Deliverables (What to Show)
June / July 2026 **Phase 1: Foundations**	• Requirements engineering (SRS drafting) • Relational database schema design (SQLAlchemy models) • User authentication (Scrypt hashing, Flask-Login) • Log forms with validation (Flask-WTF) • Journal decryption pipeline (AES-256 Fernet)	• Entity-Relationship (ER) Diagram • User Registration and Login flows • Daily logging input forms • Database migration logs (Flask-Migrate) • SQLite console showing encrypted notes
August 2026 **Phase 2: Analytics**	• Local Sentiment Analysis pipeline (NLTK VADER) • Dashboard visualization engine (Chart.js) • Interactive 3D WebGL shader orb (Three.js) • CSV data streaming export • PDF wellness report generator (ReportLab)	• Journal sentiment scoring demo (-1.0 to +1.0) • 5 interactive charts (mood, sleep, stress) • Live-deforming GPU-rendered 3D Orb • Executed CSV and PDF download exports
September 2026 **Phase 3: Machine Learning**	• Synthetic training dataset generation (generate_data.py) • ML training pipeline comparing Decision Tree/Random Forest • Model explainability (XAI) feature importance • Bias auditing & data drift monitor scripts • Model Card specification documentation	• Model performance tables (Accuracy, F1-Score) • Pickle model output and ML metrics log • Bias audit result charts (Admin dashboard) • Kolmogorov-Smirnov prediction drift charts
October 2026 **Phase 4: Security & Ops**	• CSRF protection & Login rate limiting (Flask-Limiter) • Talisman security headers & hardened session cookies • Multi-stage production Dockerfile optimization • Redis container cache integration • Automated pytest suite execution	• Security audit report (security_audit.py) • Green test suite executing in terminal (pytest) • Docker-compose start execution logs • Deployed live app on Render (using render.yaml)
November 2026 **Phase 5: Submissions**	• Compiling final project thesis/report • Detailing UML diagrams (Class, Use Case, Sequence) • PPT generation (using generate_ppt.py helper) • Internal mock vivas and review rehearsals	• Bound project document (SRS, Code, Testing) • Final slide deck presentation • Live web application demonstration

Key Highlight for Evaluators:

By structuring the project this way, you show a complete transition from a basic CRUD application (July) to an advanced data visualization tool (August), followed by an intelligent, monitored AI system (September) and a production-grade, secure cloud-deployed product (October). This progressive complexity protects you from initial review fatigue and ensures high evaluation marks at every phase.

2. The Presentation Checklist: What to Do, Show, & Say

For MCA projects, internal guides evaluate you on incremental progress. To ensure you look professional and fully prepared, structure your weekly and monthly presentations using the guidelines below.

A. Weekly Progress Reviews (Short, Agile Demos)

Weekly meetings with your guide are typically informal. Your objective is to show active coding, structured Git commits, and incremental testing. Do not try to show a complete product; show one working component.

- **What to Do:** Code in small iterations. Write unit tests for the functions you write (e.g., test your helper functions in tests/). Keep a clean Git history with descriptive commit messages.
- **What to Show:** Share a 60-second video snippet or run a quick terminal demo. Show your code diff in Git or the green pytest execution in the terminal. Teachers love seeing command-line tools like tests running successfully.
- **What to Say:** *"This week, I focused on [specific feature, e.g., the local NLTK VADER sentiment analyzer]. I integrated it into the log submission route. When a user submits a note, VADER processes the text locally, avoiding cloud API calls. I also wrote 3 unit tests to verify sentiment score boundaries (-1.0 to 1.0) and all of them pass. Next week, I will implement the database encryption for these logs."*

B. Monthly Panel Evaluations (Milestone Demos)

Monthly reviews are formal. You present to a panel of 2 or 3 professors. They grade you on architecture, database integrity, and implementation compliance.

- **What to Do:** Verify your application's integrity by running the system checks: security audits (scripts/security_audit.py), model drift detection (scripts/monitor_drift.py), and bias audits (scripts/bias_audit.py). Keep your documentation and code inline.
- **What to Show:** Present a PowerPoint slide deck (which you can generate using the scripts/generate_ppt.py utility). Demonstrate the user flow live on localhost or on your live Render URL. Show the dashboard rendering populated data and show your active logs in the Admin interface.
- **What to Say:** Focus on design patterns and technical justifications. *"During this milestone, we integrated the core Logging system with our Machine Learning Predictor. The system extracts logs, creates a 12-feature matrix representing user state, and inputs it to a Random Forest classifier. This predicts burnout risk. To ensure transparency, we display explainable AI features to the user. We also run regular bias and drift checks to maintain model accuracy over time, as displayed in the admin console."*

C. Proposed 10-Week Presentation Schedule

Week	Feature Focus	Key Review Talking Point
Week 1	SRS & ERD Draft	Database models in models.py & relational mapping.
Week 2	Auth & CRUD Logs	User sign-up and log forms validation using WTForms.
Week 3	Notes Encryption	AES-256 symmetric encryption at rest (Fernet).
Week 4	NLP Sentiment	Integrating NLTK VADER local scoring pipeline.
Week 5	Dashboard Charts	Chart.js integration, layout grids, and responsiveness.
Week 6	WebGL 3D Orb	Three.js vertex shader deforming based on daily mood.
Week 7	ML Model Training	Decision Tree / Random Forest training & classification.
Week 8	Drift & Bias Audit	Model drift monitoring and ethical bias verification.
Week 9	App Security	Flask-Limiter, HTTP security headers, CSRF audits.
Week 10	Cloud Deployment	Multi-stage Docker, PostgreSQL, and Redis on Render.

3. Academic Review Pitfalls & Viva Voce Q&A;

A. What NOT to Show & What NOT to Say

[CRITICAL] DO NOT Show Empty Dashboards: Presenting empty charts makes the project look unfinished. Pre-fill the database with realistic synthetic data using generate_data.py before every evaluation.

[CRITICAL] DO NOT Show Raw Tracebacks: A server error traceback (500 screen) in front of an external examiner causes immediate mark deductions. Keep Flask debug mode off during reviews and handle errors gracefully.

[CRITICAL] DO NOT Show Hardcoded Secrets: Never display database passwords or secret keys in your slides. Keep them in a .env file. Present a clean .env.example file.

[CRITICAL] DO NOT Say "I downloaded this ML code": Acknowledge libraries, but explain the pipeline as your own. Say: *"I implemented a scikit-learn Random Forest model, validating performance metrics locally."*

[CRITICAL] DO NOT Say "The NLP uses ChatGPT API": External cloud APIs require internet connectivity and incur billing. Highlight that DayTone runs 100% locally using NLTK VADER, ensuring absolute privacy.

[CRITICAL] DO NOT Say "I will test it later": Testing is not a final step. Point to the existing tests/ directory and demonstrate that the unit test suite can be run at any moment.

B. Anticipated Viva Voce Questions & Model Answers

Q1: Why did you choose SQLite for development and PostgreSQL for production instead of MongoDB?

Answer: Relational integrity is essential for this system. Relationships between users, logs, and audits are strictly defined. Relational foreign key constraints and cascading deletes are required, especially for GDPR data erasure. PostgreSQL provides transactional stability (ACID) in production, while SQLite provides a zero-configuration local database that uses the same SQL syntax via SQLAlchemy.

Q2: What is the 12-feature matrix used in the Burnout Predictor?

Answer: The classifier does not evaluate single-day metrics in isolation. It constructs a temporal matrix: the user's daily metrics (mood, sleep, stress), their rolling 7-day averages, mood variability (standard deviation over 7 days), consecutive bad days, weekend indicators, and the journal's VADER sentiment score. This gives the model historical context to predict burnout risk accurately.

Q3: How are sensitive user journal entries secured?

Answer: Reflections are encrypted at rest using AES-256 Fernet symmetric encryption. The key is loaded from the environment variables, meaning the database contains only encrypted ciphertexts. Even if the database is exposed, the raw text is unreadable without the environment key.

Q4: What is Model Drift and how does your project handle it?

Answer: User habits shift over time (e.g., during exam seasons or vacations), making old training data obsolete. We include a drift monitor script (monitor_drift.py) that compares active user log distributions against the baseline dataset using a Kolmogorov-Smirnov test. If drift is detected, the administrator dashboard displays an alert to trigger retraining.

Q5: Why did you choose Random Forest over a single Decision Tree?

Answer: A single Decision Tree is prone to overfitting on training data. Random Forest is an ensemble method that trains multiple decision trees on random subsets of data and aggregates their votes. This significantly reduces variance, handles feature correlations better, and yields a higher F1-score (92% vs 84%).

Q6: How do you verify that your ML model is fair and unbiased?

Answer: We implement a bias auditing script (bias_audit.py). It segments predictions across key demographics (e.g., stress and sleep groups) and calculates metric parity (discrepancy in accuracy, precision, and recall). If performance differences exceed 5%, the system flags a bias warning on the admin panel.